

DOCUMENTO DE ANALISIS: PROYECTO BUSCAMINAS

HECHO POR: JONATHAN Y MOHAMMED.

1. Investigación del Funcionamiento y reglas.

Se ha realizado un estudio exhaustivo del comportamiento del videojuego clásico para replicarlo con exactitud:

Generación y "Primer Clic Seguro": El tablero es una matriz (ej. 9x9, 16x16). Las minas no se generan (o se reubican) hasta que el jugador hace su primer clic, garantizando que nunca se pierda en el primer movimiento.

Mecánica de Números: Cada casilla segura calcula las minas en su "Vecindad de Moore" (las 8 casillas que la rodean). Muestra un número del 1 al 8.

Algoritmo de Despeje: Si una casilla tiene 0 minas adyacentes, el sistema debe revelar automáticamente todas las casillas vecinas. Si estas vecinas también son "0", el proceso se repite recursivamente hasta crear "fronteras" de números.

Mecánica de Marcado (Banderas): El jugador dispone de un número de banderas igual al número de minas. Al colocar una, el contador de minas restantes disminuye en la interfaz.

Condiciones de Fin de Partida:

Derrota (Game Over): Se revela una casilla cuyo valor interno es una mina. Se revela el resto del tablero para mostrar dónde estaban.

Victoria: Se descubre matemáticamente la última casilla que no es una mina (las banderas no son obligatorias para ganar, lo obligatorio es descubrir todas las casillas seguras).

2. Estudio de Variantes Modernas.

Buscaminas "No-Guess", es una versión donde los algoritmos de generación aseguran que el tablero siempre se pueda resolver mediante lógica.

Tableros no cuadrangulares, variantes que utilizan celdas triangulares o hexagonales.

Variantes RPG/Roguelike, juegos como Demon Crawl, que combinan las mecánicas del buscaminas con recolección de objetos, inventarios, monstruos y progresión de personajes.

Aplicación al Proyecto, aunque el proyecto se centrará en el desarrollo de la versión clásica para cumplir con los requisitos base, esta investigación servirá para futuras expansiones.

3. Identificación de Usuarios

Jugador Casual: Usuario que busca entretenimiento rápido. Interactúa con la GUI del juego, utiliza dificultades bajas y no le da gran importancia al cronómetro.

Jugador Competitivo: Usuario que busca optimizar sus tiempos. Juega en dificultades altas, requiere que la interfaz responda sin lag y su objetivo principal es registrar su nombre en el *Leaderboard* (Base de Datos).

Visitante Web: Usuario que accede a la página web promocional para conocer el juego, ver las capturas de pantalla, leer las características y descargar la aplicación.

Administrador de Sistema/BBDD: Perfil técnico, que gestiona la base de datos, asegura la integridad referencial y tiene capacidad para realizar operaciones CRUD.

4. Análisis de Requisitos Funcionales

-RF01 -Gestión de dificultades: El sistema debe proveer 3 niveles estándar: Principiante (9x9, 10 minas), Intermedio (16x16, 40 minas) y Experto (16x30, 99 minas).

-RF02 – Interfaz Dinámica: El tablero gráfico debe generarse dinámicamente en base a los arrays/matrices de la dificultad elegida.

-RF03- Eventos de ratón: El sistema debe distinguir entre clic izquierdo para descubrir y click derecho para colocar o quitar bandera.

-RF04- Cronometro: La interfaz debe incluir un temporizador visual que inicie en el primer click y se detenga al ganar o perder.

-RF05- Conexión a base de datos, tras una victoria, el sistema debe abrir un cuadro de dialogo pidiendo un alias y ejecutar una consulta SQL "INSERT" para guardar el registro.

-RF06- Web Promocional, es una web que presente el juego, sus características y una sección en ingles.

5. Análisis de Requisitos No Funcionales

RNF01 - Desarrollo estricto en Java bajo el paradigma de Programación Orientada a Objetos .

RNF02 - El código debe documentarse y el proyecto debe gestionarse mediante un repositorio en Git/GitHub, dejando evidencias de *commits*.

RNF03 - Base de Datos: Uso de un motor de BBDD relacional (ej. MariaDB), garantizando una correcta estructura de tablas.

RNF04 - La web promocional se desarrollará en HTML, CSS y JavaScript.

RNF05 - La aplicación no debe "crashear". Se debe implementar una correcta gestión de errores y excepciones (try-catch), especialmente en la conexión a la base de datos.

6. Casos de uso.

CU01 - Jugar Partida: El usuario abre la App, el sistema carga la GUI. El usuario hace clic en las casillas. El sistema gestiona la lógica (minas, números, espacios en blanco). El usuario gana o pierde.

CU02 - Consultar Ranking: El usuario pulsa el botón "Ver Puntuaciones". El sistema ejecuta un SELECT ordenado por tiempo (ASC) en la Base de Datos y muestra los resultados en una tabla gráfica.

CU03 - Cambiar Dificultad: El usuario selecciona una nueva dificultad en el menú. El sistema destruye el tablero actual, recalcula el tamaño de la matriz y renderiza un tablero nuevo limpio.

7. Historias de Usuario.

HU01: Como jugador, quiero ver un contador de banderas restantes para saber cuántas minas me faltan por localizar.

HU02: Como jugador, quiero que si hago clic en una zona vacía sin peligro, se abra toda el área de golpe para no perder tiempo haciendo clics innecesarios.

HU03: Como jugador competitivo, quiero que al ganar se guarde mi récord de tiempo en una base de datos para comparar mi habilidad con otros.

HU04: Como usuario extranjero, quiero que la web promocional tenga textos en inglés técnico para entender de qué va el proyecto antes de descargarlo.

8. Analisis Técnico Inicial.

Modelo : Clases puras de Java (ej. Casilla, Tablero) que manejan las matrices, la ubicación de las minas y la lógica matemática sin saber nada de la parte visual.

Vista (GUI): Ventanas y botones que dibujan lo que dicta el Modelo.

Persistencia: Una clase Gestor/DAO dedicada exclusivamente a establecer la conexión JDBC con la Base de Datos y lanzar las sentencias SQL.